Aubrée Romain **Beauvallet**
12/11/2026 **Semestre 3**
SLAM

Groupe 1

Bloc 2 TP 2 React

Contexte **2**
Partie 1 : Réorganisation et état centralisé **3**
Partie 2 : Suppression et changement de statut **5**
Partie 3 : Formulaire contrôlé d'ajout **7**
Partie 4 : Recherche, filtre et tri **10**
Partie 5 : Appel à une API avec useEffect **12**
Partie 6 : Navigation avec React Router **15**
Partie 7 : Finitions **18**
Pour aller plus loin **19**
Conclusion **21**

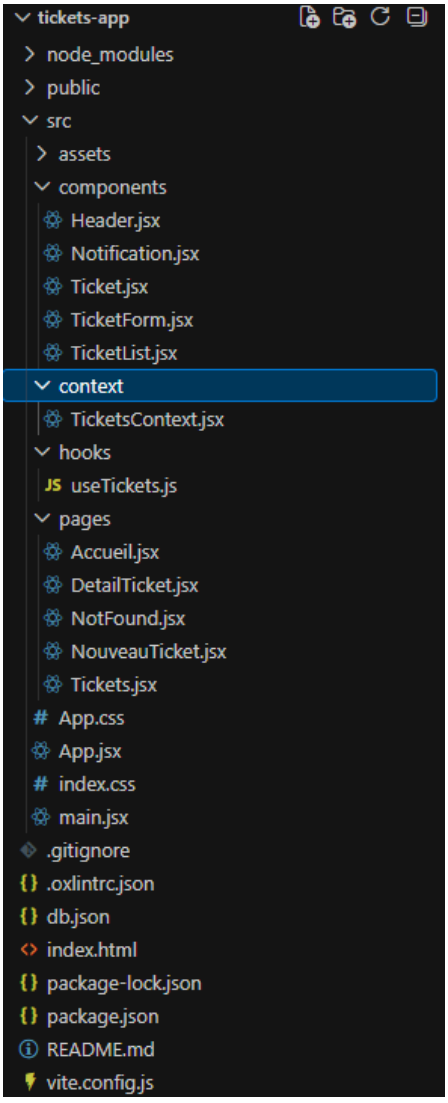
Contexte

Ce **TP2** fait suite au TP1 : le prototype **tickets-app** a convaincu WebDev Solutions, il faut maintenant le transformer en une vraie application utilisable. On va gérer un **état centralisé** (création, modification et suppression de tickets), ajouter la **recherche, le filtre et le tri**, récupérer les données depuis une **API REST** avec **useEffect**, et mettre en place la **navigation** entre plusieurs pages avec **React Router**.

*Le projet est disponible sur **GitHub** (**https://github.com/RomAub/bvl_S3_B2_TP2_React**)*

Partie 1 : Réorganisation et état centralisé

Je réorganise d'abord le projet en séparant les composants réutilisables (**components/**) des pages (**pages/**). J'ajoute aussi un hook personnalisé (**hooks/**) et un contexte (**context/**) pour centraliser les données :



La liste des tickets est stockée dans un seul endroit (l'état est « **remonté** » au plus haut niveau), puis partagée aux composants enfants ; **TicketList** reçoit les tickets en props et les affiche avec **map**. Pour éviter de passer les props de niveau en niveau, je centralise cet état dans un hook **useTickets** exposé par un **Context** React, placé autour de l'application dans **main.jsx** :

```jsx
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import { BrowserRouter } from "react-router-dom";
import "./index.css";
import App from "./App.jsx";
import { TicketsProvider } from "./context/TicketsContext.jsx";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <BrowserRouter>
      <TicketsProvider>
        <App />
      </TicketsProvider>
    </BrowserRouter>
  </StrictMode>
);
```

<u>Q1</u> Pourquoi stocke-t-on l'état dans le composant parent plutôt que dans TicketList ?

Parce que plusieurs composants ont besoin des mêmes données (la liste, la page d'accueil pour les statistiques, le formulaire, le détail). En plaçant l'état au-dessus d'eux (**source unique de vérité**), tous partagent les mêmes tickets, et une action faite dans un composant (ajout, suppression) met à jour l'affichage partout. Si l'état était dans TicketList, les autres composants ne pourraient ni y accéder ni le modifier.

<u>Q2</u> Qu'appelle-t-on le « flux de données unidirectionnel » en React ?

En React, les données circulent dans un seul sens : du **parent vers l'enfant**, via les props (ou le contexte). Un enfant ne modifie jamais directement les données du parent ; il le prévient en appelant une fonction reçue en props (un **callback**), et c'est le parent qui met à jour l'état. Les données descendent, les événements remontent.

Partie 2 : Suppression et changement de statut

J'ajoute une fonction **supprimerTicket(id)** et une fonction **changerStatut(id, nouveauStatut)** dans le hook, que je passe jusqu'au composant Ticket. Chaque carte a un bouton « Supprimer » et un menu déroulant pour changer le statut. Le point important est de ne jamais modifier le tableau directement, mais d'en créer une nouvelle version (**immutabilité**) avec **filter** et **map** :

```
5   export function useTickets() {
25    async function ajouterTicket(donnees) {
35    }
36
37    async function supprimerTicket(id) {
38      await fetch(`${API}/${id}`, { method: "DELETE" });
39      setTickets((liste) => liste.filter((t) => t.id !== id));
40    }
41
42    async function changerStatut(id, nouveauStatut) {
43      await fetch(`${API}/${id}`, {
44        method: "PATCH",
45        headers: { "Content-Type": "application/json" },
46        body: JSON.stringify({ statut: nouveauStatut }),
47      });
48      setTickets((liste) =>
49        liste.map((t) => (t.id === id ? { ...t, statut: nouveauStatut } : t))
50      );
51    }
52
53    async function modifierTicket(id, modifs) {
54      await fetch(`${API}/${id}`, {
55        method: "PATCH",
56        headers: { "Content-Type": "application/json" },
57        body: JSON.stringify(modifs),
58      });
59      setTickets((liste) =>
60        liste.map((t) => (t.id === id ? { ...t, ...modifs } : t))
61      );
62    }
63
64    return {
65      tickets,
66      chargement,
67      erreur,
68      ajouterTicket,
69      supprimerTicket,
70      changerStatut,
71      modifierTicket,
72    };
73  }
74
```

Dans l'interface, le bouton « Supprimer » demande une confirmation avant d'agir, et le statut se change directement depuis la carte :



<u>Q3</u> Pourquoi ne faut-il jamais utiliser push, splice ou modifier directement un objet du state ?

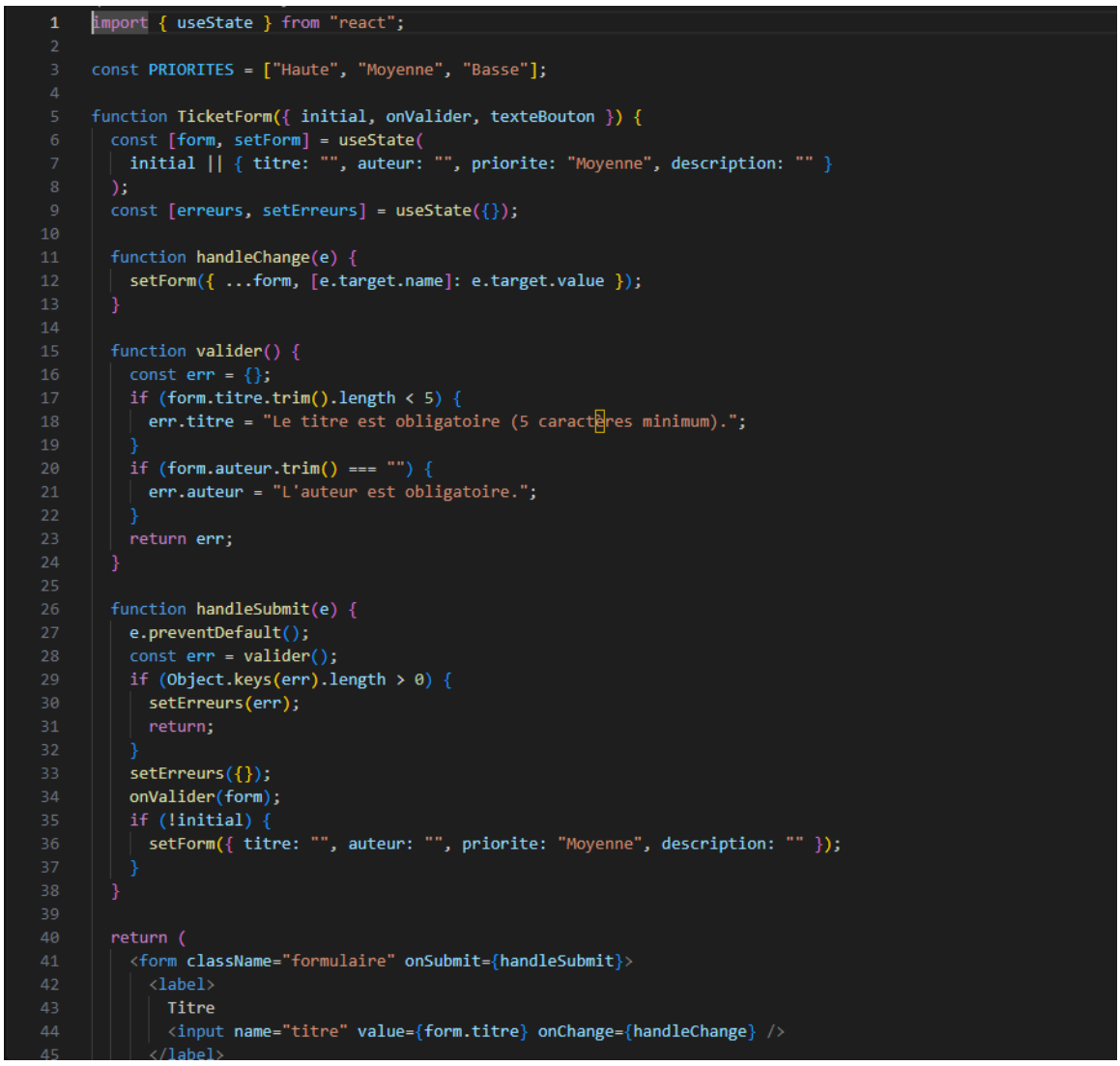
Parce que React détecte les changements en comparant les **références** (l'adresse en mémoire) de l'ancien et du nouveau state. **push** ou **splice** modifient le tableau existant sans changer sa référence : React ne « voit » donc pas le changement et ne réaffiche pas. Il faut créer un nouveau tableau (avec filter, map ou l'opérateur ...), ce qui donne une nouvelle référence et déclenche le rendu.

<u>Q4</u> Que fait l'opérateur de décomposition ...t ?

L'opérateur de décomposition (**spread**) copie toutes les propriétés d'un objet (ou les éléments d'un tableau) dans un nouvel objet/tableau. Par exemple **{ ...t, statut: nouveauStatut }** crée une copie du ticket t avec toutes ses propriétés, mais en remplaçant statut. Cela permet de modifier une donnée sans toucher à l'originale (immutabilité).

Partie 3 : Formulaire contrôlé d'ajout

Je crée un composant **TicketForm** avec des champs **contrôlés** (chaque champ a un value lié au state et un onChange). Je valide le formulaire : le **titre** est obligatoire (5 caractères minimum) et l'**auteur** est obligatoire, sinon un message d'erreur s'affiche sous le champ concerné. À la validation, le ticket est ajouté avec un id unique (**Date.now()**) et le statut « Ouvert », puis le formulaire est réinitialisé :



```jsx
import { useState } from "react";

const PRIORITES = ["Haute", "Moyenne", "Basse"];

function TicketForm({ initial, onValider, texteBouton }) {
  const [form, setForm] = useState(
    initial || { titre: "", auteur: "", priorite: "Moyenne", description: "" }
  );
  const [erreurs, setErreurs] = useState({});

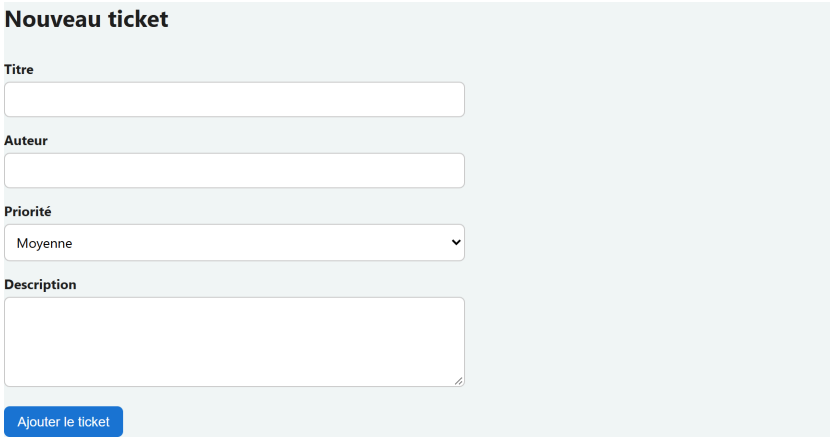
  function handleChange(e) {
    setForm({ ...form, [e.target.name]: e.target.value });
  }

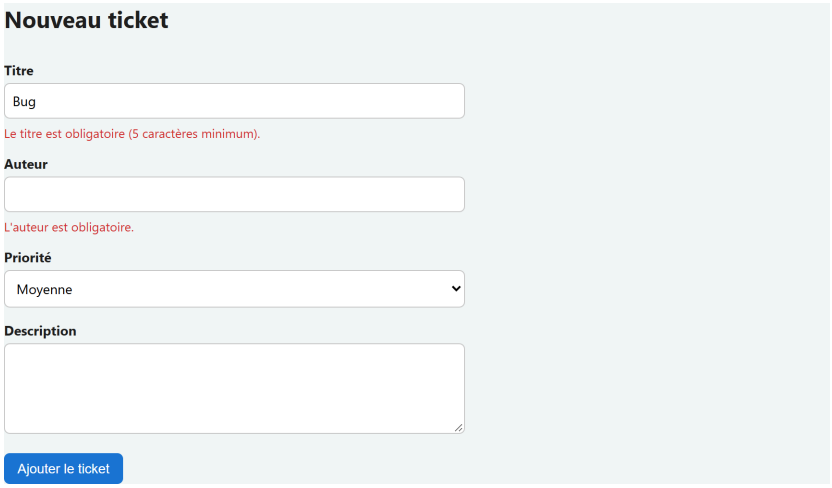
  function valider() {
    const err = {};
    if (form.titre.trim().length < 5) {
      err.titre = "Le titre est obligatoire (5 caractères minimum).";
    }
    if (form.auteur.trim() === "") {
      err.auteur = "L'auteur est obligatoire.";
    }
    return err;
  }

  function handleSubmit(e) {
    e.preventDefault();
    const err = valider();
    if (Object.keys(err).length > 0) {
      setErreurs(err);
      return;
    }
    setErreurs({});
    onValider(form);
    if (!initial) {
      setForm({ titre: "", auteur: "", priorite: "Moyenne", description: "" });
    }
  }

  return (
    <form className="formulaire" onSubmit={handleSubmit}>
      <label>
        Titre
        <input name="titre" value={form.titre} onChange={handleChange} />
      </label>
```

Voici le formulaire vide, puis le même formulaire quand la validation échoue (titre trop court et auteur manquant) :





Q5 Quelle différence entre composant contrôlé et non contrôlé ?

Un composant **contrôlé** a sa valeur pilotée par le state React (value + onChange) : React est la source de vérité. Un composant **non contrôlé** laisse le DOM gérer sa valeur, qu'on récupère seulement au besoin avec une référence (**ref**). Les formulaires contrôlés sont plus simples à valider et à réinitialiser.

Q6 À quoi sert e.preventDefault() ? Que se passe-t-il sans lui ?

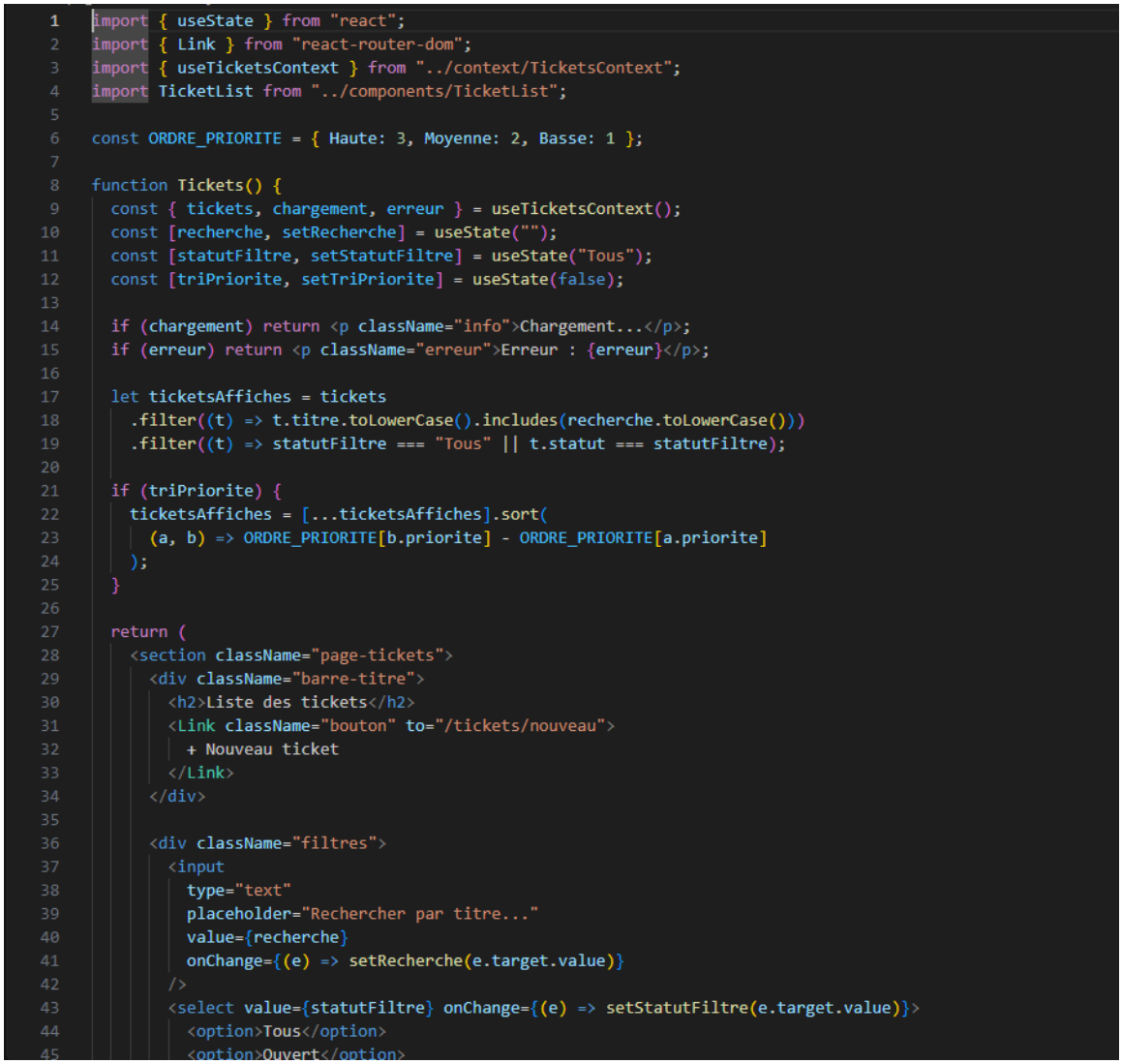
e.preventDefault() empêche le comportement par défaut du navigateur lors de la soumission du formulaire, c'est-à-dire le **rechargement complet de la page**. Sans lui, la page se rechargerait, l'application React repartirait de zéro et on perdrait l'état (les tickets, les champs saisis).

<u>Q7</u> Expliquez la syntaxe [e.target.name]: e.target.value.

C’est un « **nom de propriété calculé** » : les crochets permettent d’utiliser une variable comme clé d’objet. **e.target.name** est le nom du champ modifié (ex. « titre ») et **e.target.value** sa nouvelle valeur. On met donc à jour dynamiquement le bon champ du state avec un seul gestionnaire handleChange commun à tous les champs, au lieu d’en écrire un par champ.

Partie 4 : Recherche, filtre et tri

Dans la page des tickets, j'ajoute un **champ de recherche** par titre (insensible à la casse), un **filtre par statut** (Tous / Ouvert / En cours / Résolu), un **tri par priorité** (Haute > Moyenne > Basse) et un **compteur** « X ticket(s) affiché(s) sur Y ». La liste affichée n'est pas stockée dans un state : elle est **recalculée à chaque rendu** à partir de la liste complète :



```jsx
import { useState } from "react";
import { Link } from "react-router-dom";
import { useTicketsContext } from "../context/TicketsContext";
import TicketList from "../components/TicketList";

const ORDRE_PRIORITE = { Haute: 3, Moyenne: 2, Basse: 1 };

function Tickets() {
  const { tickets, chargement, erreur } = useTicketsContext();
  const [recherche, setRecherche] = useState("");
  const [statutFiltre, setStatutFiltre] = useState("Tous");
  const [triPriorite, setTriPriorite] = useState(false);

  if (chargement) return <p className="info">Chargement...</p>;
  if (erreur) return <p className="erreur">Erreur : {erreur}</p>;

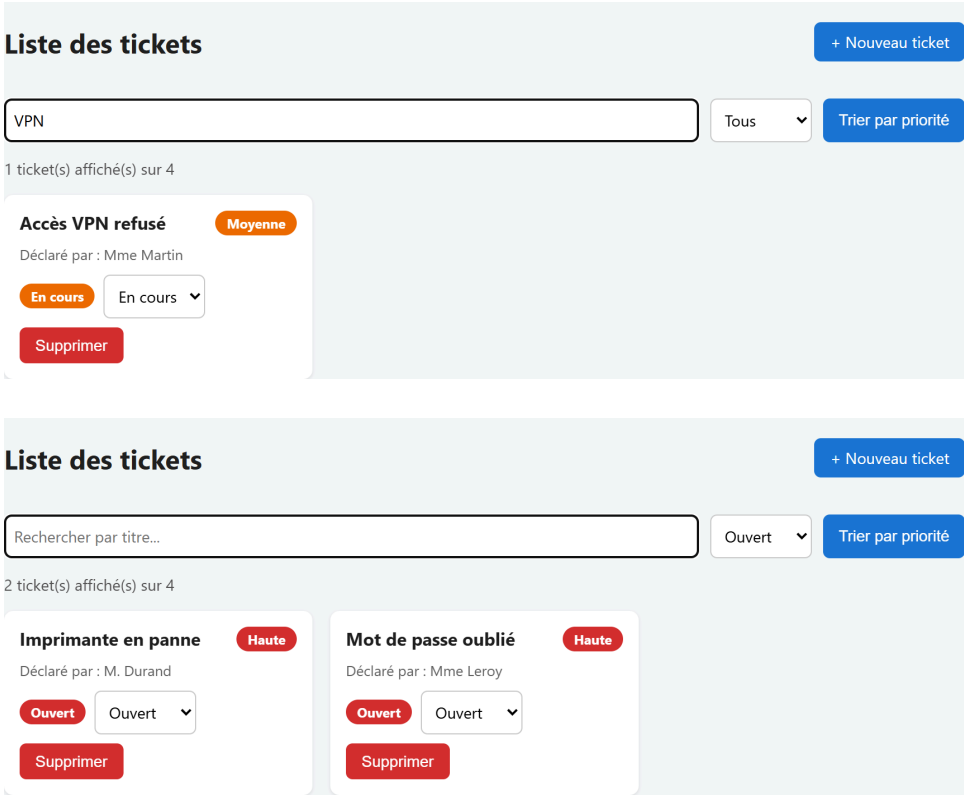
  let ticketsAffiches = tickets
    .filter((t) => t.titre.toLowerCase().includes(recherche.toLowerCase()))
    .filter((t) => statutFiltre === "Tous" || t.statut === statutFiltre);

  if (triPriorite) {
    ticketsAffiches = [...ticketsAffiches].sort(
      (a, b) => ORDRE_PRIORITE[b.priorite] - ORDRE_PRIORITE[a.priorite]
    );
  }

  return (
    <section className="page-tickets">
      <div className="barre-titre">
        <h2>Liste des tickets</h2>
        <Link className="bouton" to="/tickets/nouveau">
          + Nouveau ticket
        </Link>
      </div>

      <div className="filtres">
        <input
          type="text"
          placeholder="Rechercher par titre..."
          value={recherche}
          onChange={(e) => setRecherche(e.target.value)}
        />
        <select value={statutFiltre} onChange={(e) => setStatutFiltre(e.target.value)}>
          <option>Tous</option>
          <option>Ouvert</option>
```

La recherche filtre en temps réel (ici « VPN »), et le filtre par statut combiné au compteur montre bien le nombre de résultats :



<u>Q8</u> Pourquoi vaut-il mieux calculer la liste filtrée plutôt que la stocker dans un second state ?

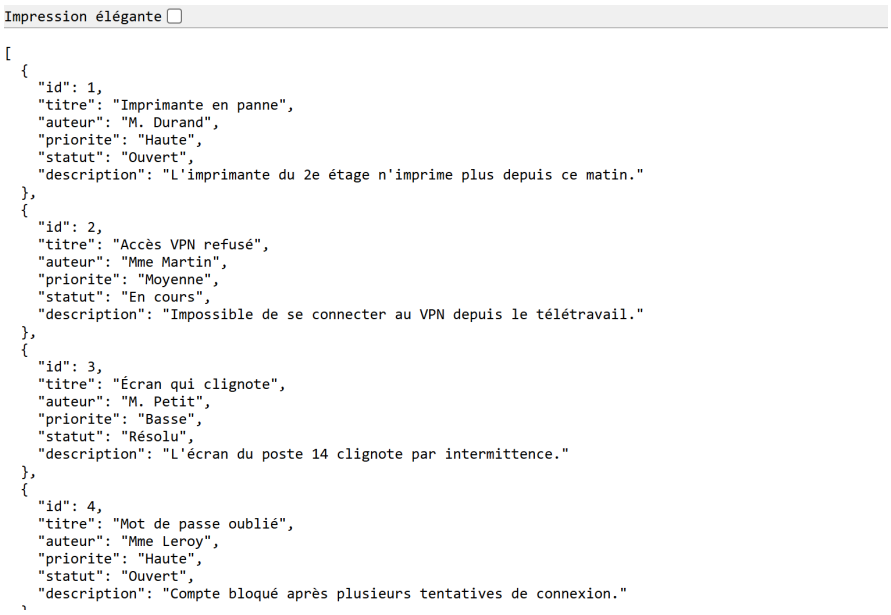
Parce que la liste filtrée est une donnée **dérivée** : elle se déduit de la liste complète et des critères (recherche, filtre, tri). La stocker dans un second state obligerait à la resynchroniser manuellement à chaque changement, avec un risque de bugs (données incohérentes). En la recalculant à chaque rendu, elle est toujours à jour automatiquement.

Partie 5 : Appel à une API avec useEffect

Pour rendre les données persistantes, je mets en place une API locale avec **json-server**. Je l'installe, je crée un fichier **db.json** à la racine, puis j'ajoute le script **"api": "json-server --watch db.json --port 3001"** dans package.json et je le lance dans un second terminal avec **npm run api** :



L'API est accessible dans le navigateur à l'adresse **http://localhost:3001/tickets**, qui renvoie les tickets au format JSON :



Ensuite, je charge les tickets depuis l'API avec **useEffect** au démarrage, en gérant un état de **chargement** et un état d'**erreur**. L'ajout, la suppression et le changement de statut deviennent persistants grâce aux requêtes **POST, DELETE et PATCH**. Toute cette logique est encapsulée dans le hook **useTickets** :

```jsx
import { useState, useEffect } from "react";

const API = "http://localhost:3001/tickets";

export function useTickets() {
  const [tickets, setTickets] = useState([]);
  const [chargement, setChargement] = useState(true);
  const [erreur, setErreur] = useState(null);

  useEffect(() => {
    async function charger() {
      try {
        const reponse = await fetch(API);
        if (!reponse.ok) throw new Error("Erreur " + reponse.status);
        setTickets(await reponse.json());
      } catch (e) {
        setErreur(e.message);
      } finally {
        setChargement(false);
      }
    }
    charger();
  }, []);

  async function ajouterTicket(donnees) {
    const nouveau = { ...donnees, id: Date.now(), statut: "Ouvert" };
    const reponse = await fetch(API, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(nouveau),
    });
    const cree = await reponse.json();
    setTickets((liste) => [...liste, cree]);
    return cree;
  }

  async function supprimerTicket(id) {
    await fetch(`${API}/${id}`, { method: "DELETE" });
    setTickets((liste) => liste.filter((t) => t.id !== id));
  }

  async function changerStatut(id, nouveauStatut) {
    await fetch(`${API}/${id}`, {
      method: "PATCH",
      headers: { "Content-Type": "application/json" },
```

L'application se lance toujours avec **npm run dev** (dans un premier terminal), pendant que l'API tourne dans le second :

```
PS C:\Users\Gobeul le Squelette\Documents\ZONE DE TAF ST\B2 TP2 REACT\tickets-app> npm run dev
  VITE v8.3.6  ready in 693 ms

  ➜ Local:   http://localhost:5173/
  ➜ Network: use --host to expose
  ➜ press h + enter to show help
```

Q9 Que signifie le tableau de dépendances [] de useEffect ? Que se passe-t-il s'il est omis ?

Le tableau **[]** vide signifie que l'effet ne s'exécute **qu'une seule fois**, juste après le premier affichage du composant (le montage). Si on l'omet, l'effet s'exécute après chaque rendu : comme il met à jour le state (donc provoque un nouveau rendu), on tombe dans une **boucle infinie** de requêtes.

Q10 Citez les quatre verbes HTTP principaux et leur correspondance CRUD.

GET = lire (Read), **POST** = créer (Create), **PUT/PATCH** = modifier (Update), **DELETE** = supprimer (Delete). Ce sont les quatre opérations de base du CRUD.

Q11 Pourquoi useEffect ne peut-il pas recevoir directement une fonction async ?

Parce qu'une fonction **async** renvoie toujours une **Promise**, alors que useEffect attend soit rien, soit une fonction de nettoyage. Si on lui passait une fonction async, il recevrait une Promise à la place de la fonction de nettoyage. On définit donc une fonction async à l'intérieur de useEffect, puis on l'appelle.

Partie 6 : Navigation avec React Router

J'installe **react-router-dom** et j'enveloppe l'application dans **<BrowserRouter>** (dans main.jsx). Je définis ensuite les **routes** dans App.jsx : la page d'accueil, la liste, le formulaire d'ajout, le détail d'un ticket, et une page 404 :

```jsx
import { Routes, Route } from "react-router-dom";
import Header from "./components/Header";
import Notification from "./components/Notification";
import Accueil from "./pages/Accueil";
import Tickets from "./pages/Tickets";
import NouveauTicket from "./pages/NouveauTicket";
import DetailTicket from "./pages/DetailTicket";
import NotFound from "./pages/NotFound";
import "./App.css";

function App() {
  return (
    <div className="app">
      <Header />
      <Notification />
      <main className="contenu">
        <Routes>
          <Route path="/" element={<Accueil />} />
          <Route path="/tickets" element={<Tickets />} />
          <Route path="/tickets/nouveau" element={<NouveauTicket />} />
          <Route path="/tickets/:id" element={<DetailTicket />} />
          <Route path="*" element={<NotFound />} />
        </Routes>
      </main>
    </div>
  );
```

Les routes correspondent aux pages suivantes :

Chemin	Page
/	Accueil (présentation et statistiques)
/tickets	Liste, recherche et filtres
/tickets/nouveau	Formulaire d'ajout
/tickets/:id	Détail d'un ticket
*****	Page 404 (route inconnue)

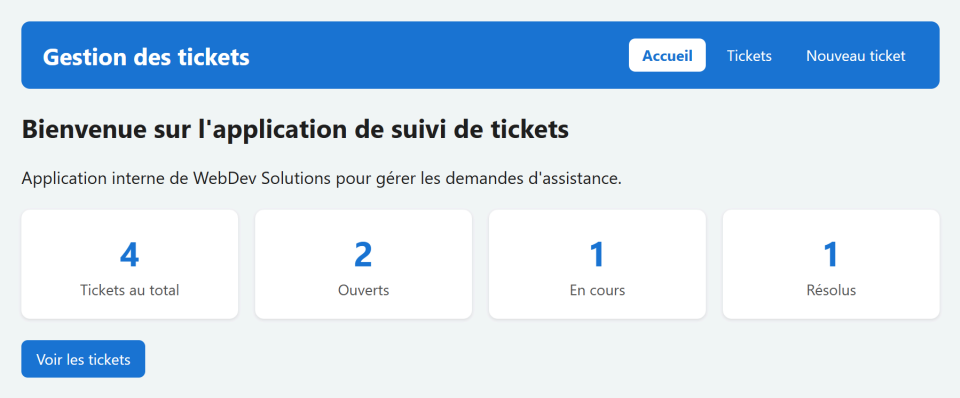
J’ajoute un menu de navigation avec **<NavLink>** dans le Header (le lien de la page active est mis en évidence) :

```jsx
import { NavLink } from "react-router-dom";

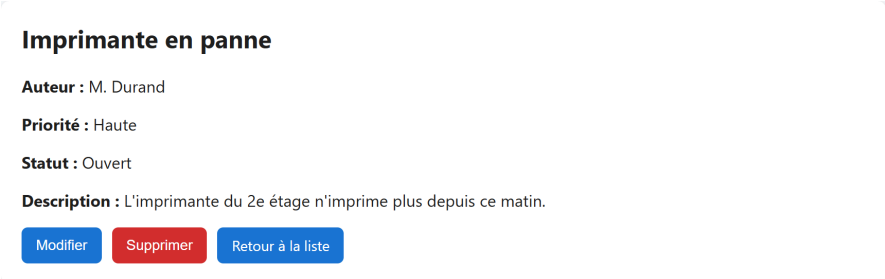
function Header() {
  return (
    <header className="header">
      <h1>Gestion des tickets</h1>
      <nav className="nav">
        <NavLink to="/" end>
          Accueil
        </NavLink>
        <NavLink to="/tickets">Tickets</NavLink>
        <NavLink to="/tickets/nouveau">Nouveau ticket</NavLink>
      </nav>
    </header>
  );
}

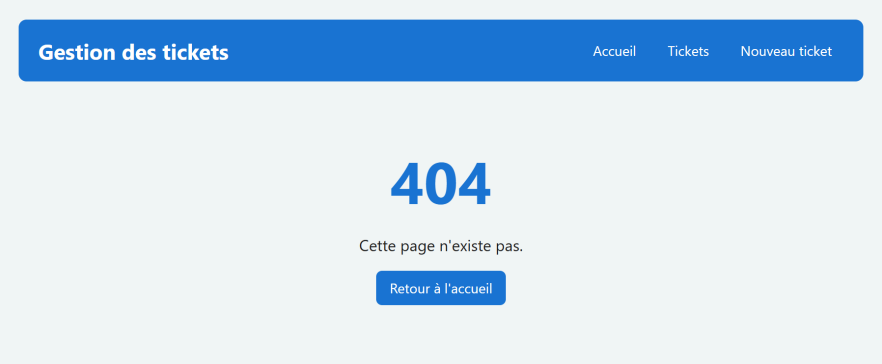
export default Header;
```

La page d’accueil présente l’application et des statistiques, et chaque page est accessible depuis le menu :



Dans la page détail, je récupère l’identifiant de l’URL avec **useParams()** pour afficher le bon ticket ; après l’ajout, je redirige vers la liste avec **useNavigate()**. Une adresse inconnue affiche la page 404 :



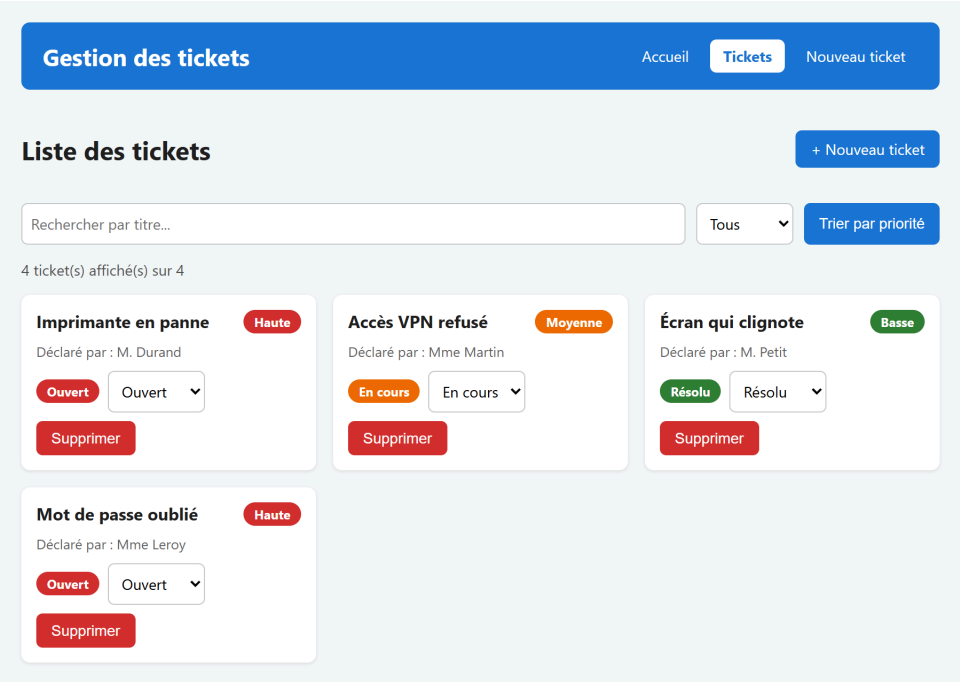


<u>Q12</u> Quelle différence entre <a href> et <Link> ? Quel est l'intérêt pour une SPA (Single Page Application) ?

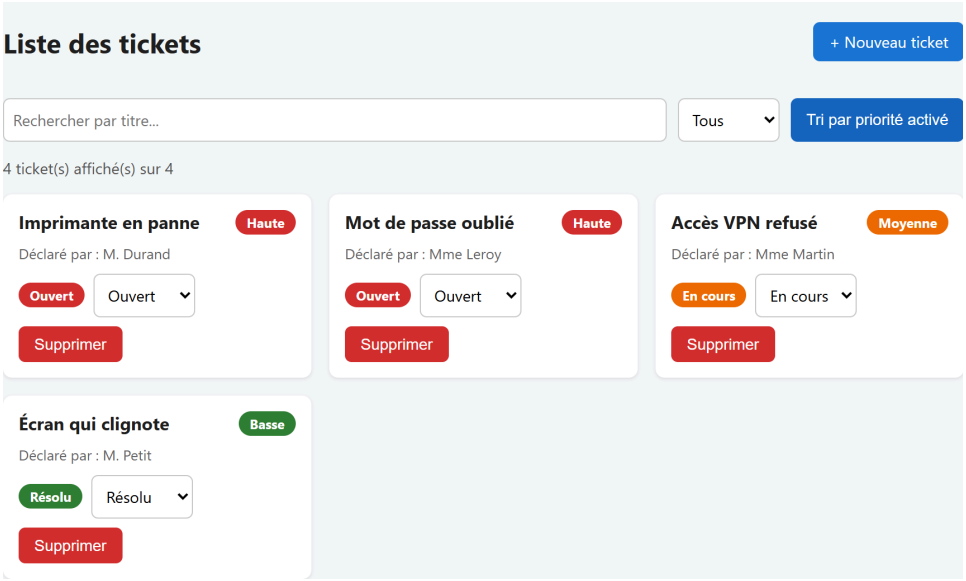
Une balise **<a href>** recharge toute la page (nouvelle requête au serveur), ce qui réinitialise l'application. **<Link>** (ou <NavLink>) de React Router intercepte le clic et change seulement la partie affichée côté client, sans recharger la page. Pour une **SPA**, c'est essentiel : la navigation est instantanée et l'état de l'application (les tickets déjà chargés) est conservé.

Partie 7 : Finitions

Pour finir, la page d'accueil affiche des **statistiques** (nombre total de tickets et nombre par statut). J'ai soigné la mise en forme CSS avec des **badges de couleur** pour les priorités et les statuts (rouge, orange, vert), une mise en page **responsive**, et j'ai nettoyé le code (pas de console.log, composants nommés clairement) :

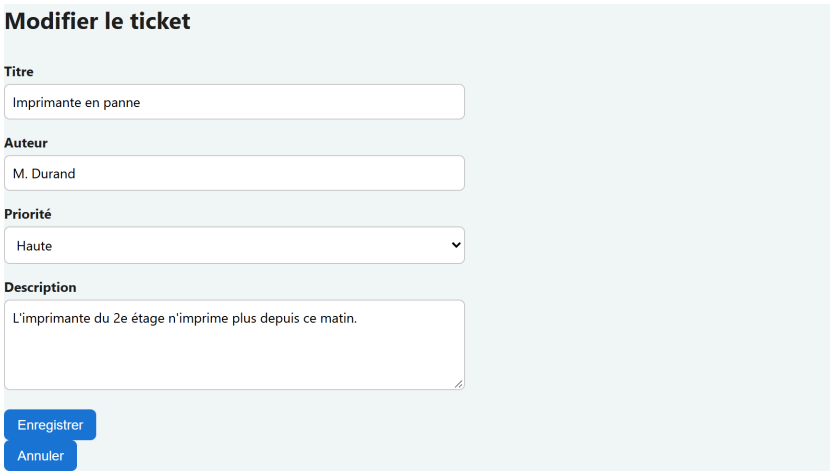


Chaque ticket s'affiche sous forme de carte avec sa priorité et son statut en badges colorés, et le tri par priorité remonte les tickets les plus urgents :



Pour aller plus loin

J'ai ajouté plusieurs améliorations. D'abord l'**édition d'un ticket existant** : depuis la page détail, un bouton « Modifier » ouvre le même formulaire pré-rempli avec les valeurs du ticket :



Ensuite, j'ai créé un hook personnalisé **useTickets()** qui encapsule tous les appels à l'API, et j'utilise **useContext** (un TicketsContext) pour partager la liste des tickets à tous les composants sans passer les props de niveau en niveau :



```jsx
import { createContext, useContext, useState, useCallback } from "react";
import { useTickets } from "../hooks/useTickets";

const TicketsContext = createContext(null);

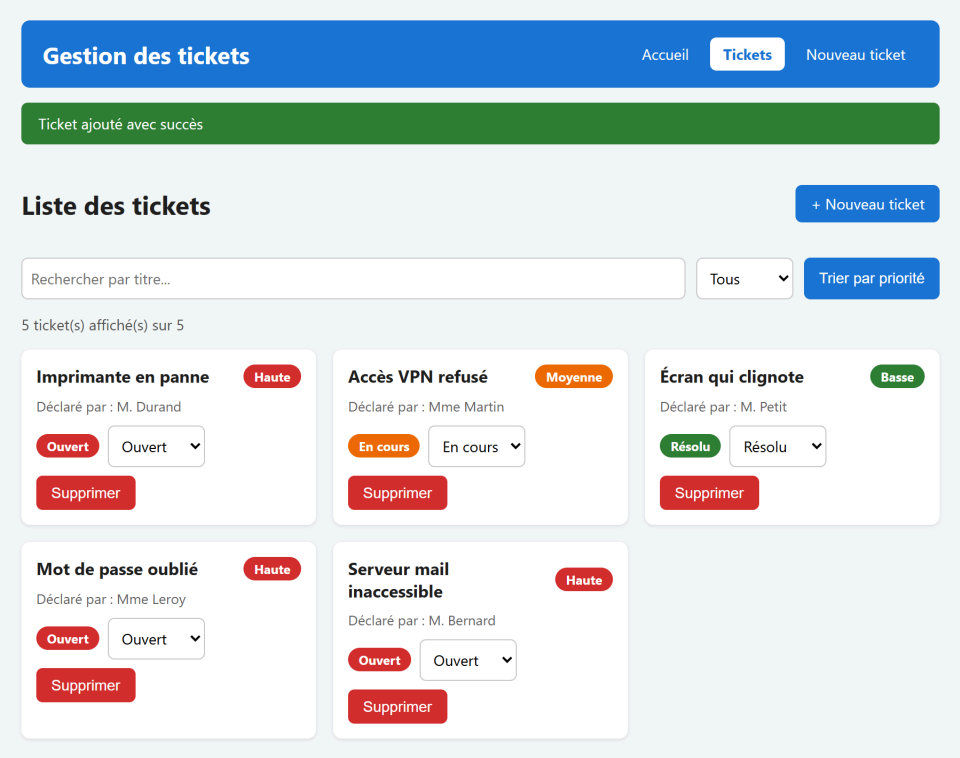
export function TicketsProvider({ children }) {
  const api = useTickets();
  const [message, setMessage] = useState("");

  const notifier = useCallback((texte) => {
    setMessage(texte);
    setTimeout(() => setMessage(""), 3000);
  }, []);

  return (
    <TicketsContext.Provider value={{ ...api, message, notifier }}>
      {children}
    </TicketsContext.Provider>
  );
}

export function useTicketsContext() {
  return useContext(TicketsContext);
}
```

Enfin, j'ai ajouté une **confirmation avant chaque suppression** (vue plus haut) et des **notifications de succès** : après un ajout, un message vert apparaît et la liste est mise à jour :



Conclusion

Ce TP2 m'a permis de transformer le prototype du TP1 en une vraie application : gestion d'un **état centralisé et immuable**, **formulaire contrôlé** avec validation, recherche/filtre/tri, récupération et persistance des données via une **API REST** (json-server) avec **useEffect** et les verbes HTTP, et **navigation** entre plusieurs pages avec **React Router**. J'ai aussi ajouté l'édition, un hook personnalisé, un contexte, une confirmation de suppression et des notifications.

Principales **difficultés rencontrées** : la version récente de json-server (v1) ne reconnaît plus l'option **--watch**, j'ai donc installé la version 0.17.4 pour correspondre au sujet ; l'erreur « **Failed to fetch** » quand j'oubliais de lancer npm run api dans un second terminal ; et le risque de **boucle infinie** avec useEffect, résolu en pensant bien au tableau de dépendances [].